Rapport Passion Lecture

Introduction 2
Analyse 2
Planification de toutes les tâches à réaliser (1 pages) 9
Structure du code 9
Réalisation 10
Explication sur chaque fonctionnalité technique demandée. 10
Organisation du groupe et gestion du code 11
Conclusion 11
Conclusion personnelle 12
Thomas Peltier 12
Thomas Reynaud 12
Ryan Läuppi 12
Critique constructive sur la planification du projet (½ page) 12
Webographie / Bibliographique / Glossaire 12

Introduction

Le projet *Passion Lecture* consiste à développer le frontend complet d'une application web destinée aux passionnés de lecture. L'objectif principal est de permettre aux utilisateurs de consulter, ajouter, modifier et commenter des ouvrages littéraires, tout en appliquant les compétences acquises dans le module 294. Comme indiqué dans le cahier des charges, *« ce projet vise à mettre en œuvre les connaissances apprises dans le module 294 »*, et il constitue la première partie d'un ensemble, puisque le backend sera réalisé ultérieurement dans le cadre du module 295.

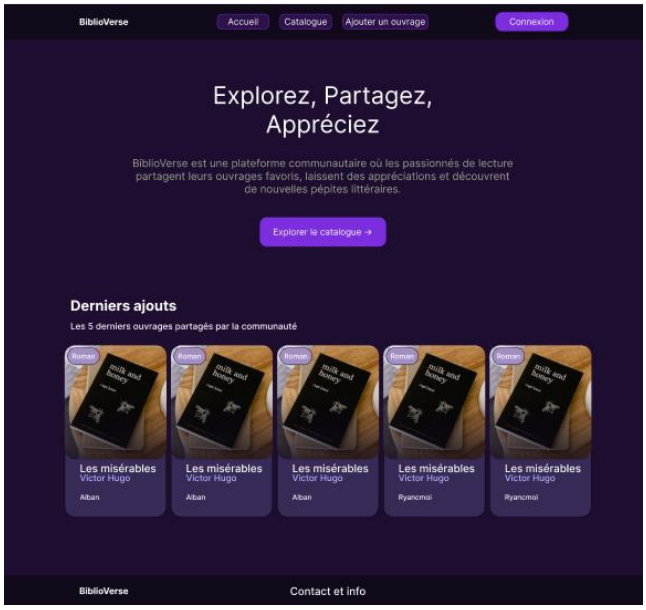
L'application doit être construite avec Vue.js, Vue Router, et utiliser My JSON Server pour simuler une API REST, car aucun backend réel n'est disponible. Le site doit proposer plusieurs pages essentielles : une page d'accueil présentant le projet et les cinq derniers ouvrages ajoutés, une page listant les ouvrages par catégorie, ainsi que des pages permettant d'ajouter, modifier ou supprimer un livre pour les utilisateurs connectés. Une vue détaillée doit également permettre d'afficher toutes les informations d'un ouvrage et d'y ajouter une appréciation ou un commentaire.

Analyse

Présentation des maquettes (2 pages)

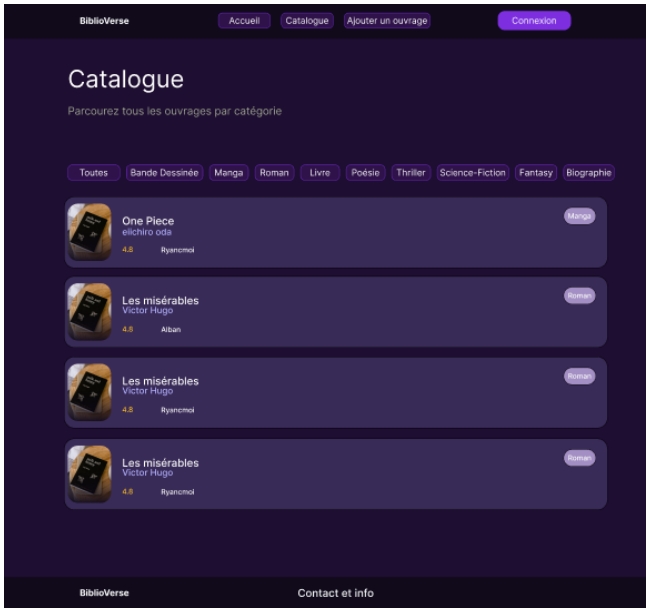
Page d'accueil :

Pour la page d'accueil, nous avons respecté les consignes du cahier des charges. Nous avons intégré une présentation du site, expliquant son utilité et son fonctionnement, ainsi que l'affichage des cinq derniers ouvrages ajoutés. Afin de rendre l'interface plus agréable et moderne, nous avons également conçu un design attractif, accompagné d'un header permettant de naviguer facilement entre les différentes pages, et d'un footer contenant le lien vers la page de contact et des infos sur les créateurs.



Page liste des ouvrages :

La page catalogue est une page qui liste les ouvrages conseiller par la communauté de passionner on peut les filtrer par thèmes de livre comme manga, Roman ou Thriller. Nous avons aussi les informations sur le nom du livre, le nom de l'auteur, la moyenne de la note donner par la communauté, le thème du livre et le pseudo de la personne qui la conseiller.

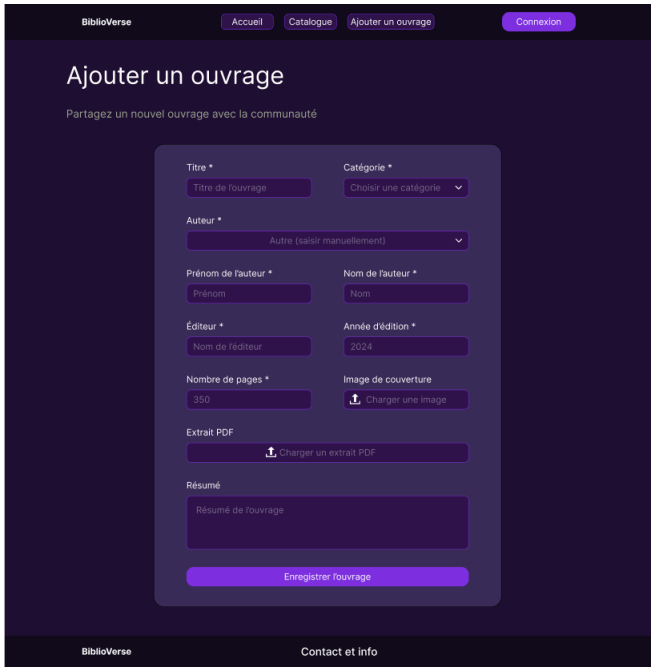


Page ajout d'un ouvrage :

La page ajout d'un ouvrage permet d'ajouter un ouvrage que l'utilisateur a apprécié et qu'il veut partager à la communauté. Nous avons donc ajouté un formulaire qui respecte les critères du cahier des charges.

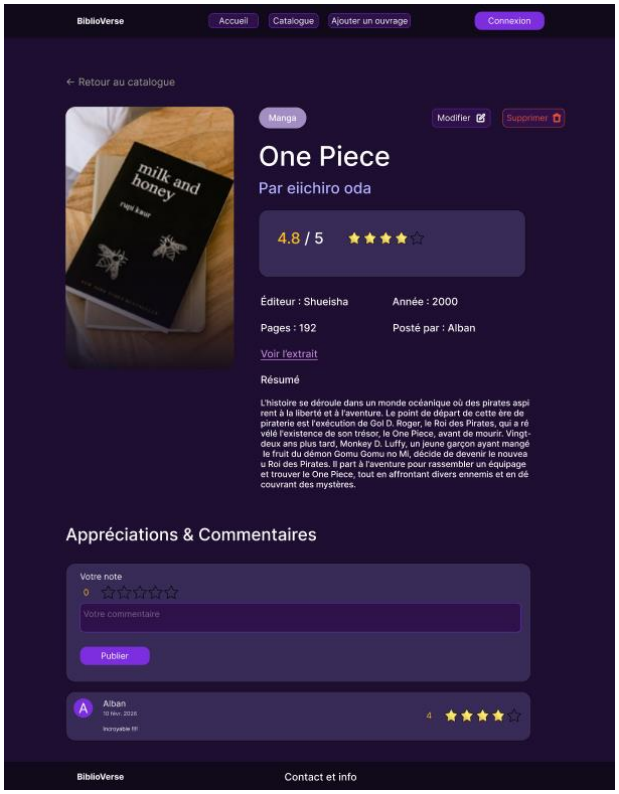
Il faut fournir les informations suivantes, qui sont obligatoires : le titre, la catégorie, l'auteur. S'il a déjà été renseigné auparavant, on peut retrouver son nom dans une liste déroulante ; sinon, il faut entrer son nom et son prénom dans les champs respectifs. Il faut aussi renseigner l'éditeur, l'année d'édition et le nombre de pages.

Pour les champs facultatifs, il y a l'image de couverture, l'extrait en PDF et le résumé du livre. Ensuite, il suffit d'enregistrer l'ouvrage et il est ajouté au site.



Page détaille d’un ouvrage :

La page détail d’un ouvrage permet de voir les informations sur un ouvrage qu’un utilisateur a ajouté. Les informations que l’on peut voir sur cette page sont les suivantes : le titre, l’auteur, la catégorie, la moyenne de la note, l’éditeur, l’année d’édition, le nombre de pages, le pseudo de l’utilisateur qui l’a posté, le résumé, l’image de l’ouvrage, un lien cliquable qui permet de télécharger l’extrait PDF, ainsi qu’une zone d’appréciations et de commentaires où les utilisateurs peuvent publier un avis et une note. Il est possible pour l’utilisateur connecté qui à poster cet ouvrage de le modifier ou de le supprimer.



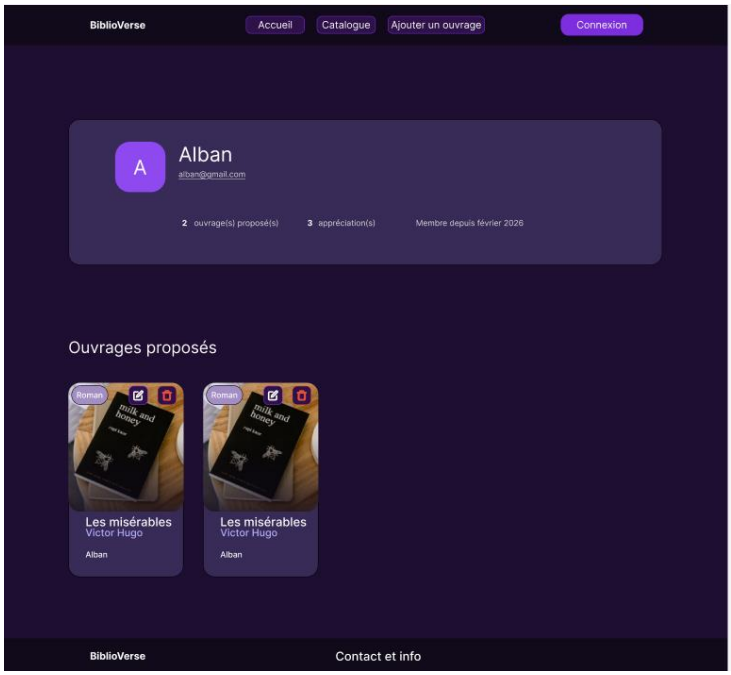
Page contact et info :

La page Contact et info permet d'avoir des informations sur les créateurs du site. Sur cette page, on peut voir une image qui représente chacun des créateurs, leurs noms ainsi qu'une adresse e-mail pour les contacter.

Page utilisateur :

La page utilisateur est la page personnelle de l'utilisateur. Elle est accessible uniquement si l'utilisateur est connecté.

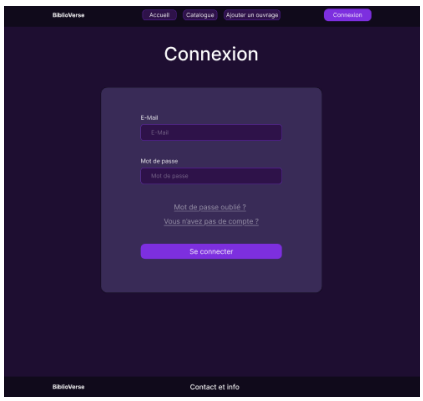
Sur cette page, on peut voir notre pseudo, notre e-mail, le nombre d'ouvrages proposés, le nombre d'appréciations sur nos ouvrages, la date d'arrivée (création du compte) ainsi que tous les ouvrages que nous avons proposés, avec la possibilité de les supprimer ou de les modifier.



Page connexion :

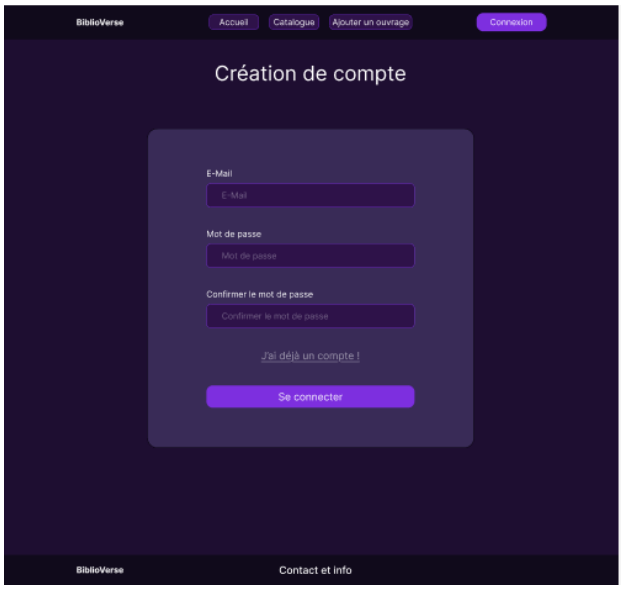
La page connexion permet à un utilisateur de se connecter. Sur cette page, il y a un formulaire où l'utilisateur doit renseigner son e-mail et son mot de passe. Ensuite, il peut appuyer sur le bouton pour se connecter.

Il y a également un lien pour un mot de passe oublié ainsi qu'un lien pour créer un compte s'il n'en a pas déjà un.



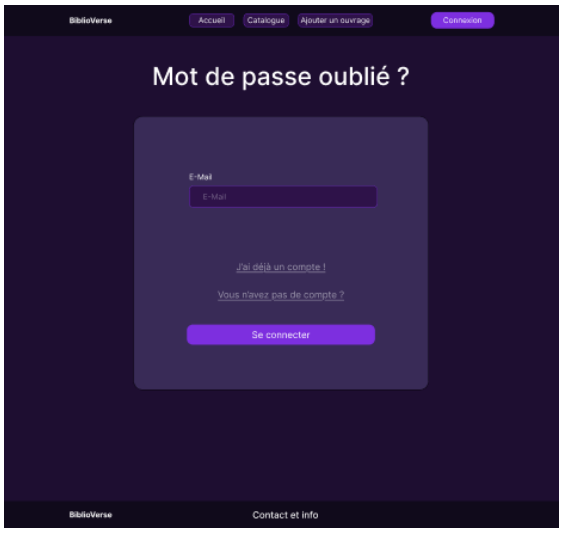
Page connexion :

La page création de compte permet à l'utilisateur de créer un compte. Un formulaire sur cette page demande de renseigner l'E-mail de l'utilisateur et un mot de passe, qui doit également être confirmé. Il y a aussi un lien pour revenir à la page connexion si l'utilisateur ne souhaite pas rester sur cette page.

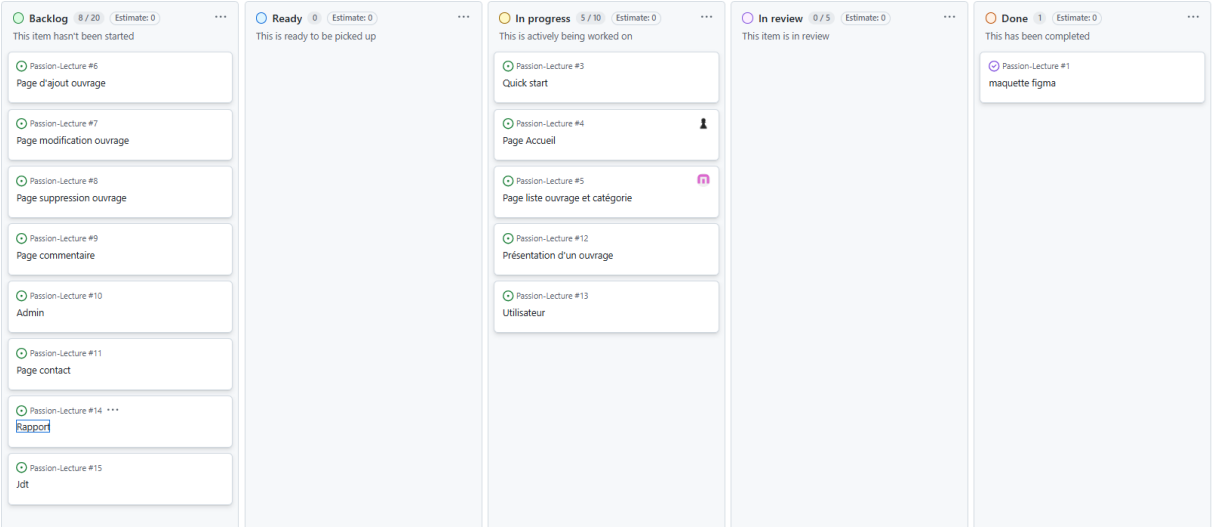


Page mot de passe oublié :

La page mot de passe oublié permet à l'utilisateur de renseigner son e-mail afin de récupérer son mot de passe. Il y a également des liens permettant de revenir sur l'une des deux pages de connexion.



Planification de toutes les tâches à réaliser (1 pages)



Pour la planification de toutes les tâches, nous avons décidé d'utiliser la méthode Kanban dans GitHub Project. Nous avons pris les demandes du cahier des charges et les avons découpées en petites tâches, que nous avons ensuite réparties par pages à réaliser (page d'accueil, catalogue, ajout d'ouvrage, etc.) et par éléments annexes comme le rapport ou le JDT.

Le tableau Kanban a été structuré avec des colonnes telles que À faire, En cours, À tester et Terminé, ce qui nous a permis de suivre clairement l'avancement de chaque tâche. Chaque tâche était attribuée à un membre du groupe en fonction de ses compétences, et nous avons aussi défini des priorités pour nous assurer que les fonctionnalités essentielles soient développées en premier.

Nous avons mis à jour le tableau régulièrement pour refléter l'état réel du projet et nous avons pu ajuster l'ordre des tâches lorsque de nouvelles idées ou contraintes apparaissaient. Ce suivi a permis de mieux coordonner le travail en équipe, de ne rien oublier et de s'assurer que toutes les étapes du projet soient couvertes.

Structure du code

Le projet a été initialisé via la commande npm create vue@latest. La structure repose sur un fichier racine, App.vue, qui définit la structure globale de la nav, main, footer. L'application fonctionne comme une SPA (Single Page Application), le contenu dynamique est apporté dans le Main via des Views (HomeView.vue, etc..). Ces vues agissent comme des composants qui assemblent des sous-composants, comme Accueil.vue. Le changement de page est géré par le fichier index.js qui agit comme un

router qui définit le chemin pour chaque view et les commponents qu view et les qu'elle utilise. Chaque composant regroupe sa structure,html(template), script(js), css. Pour la gestion des données, nous avons centralisé les appels HTTP dans un service dédié qui s'appelle bookServices.js. Ce dernier utilise la bibliothèque Axios pour communiquer avec notre fichier JSON.

```
import axios from 'axios'

const apiClient = axios.create({
  baseURL: 'http://localhost:3000/',
  withCredentials: false,
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json',
  },
})

export const getBooks = () => {
  const books = apiClient.get('books')
  return books
}

export const getBookById = (id) => {
  const books = apiClient.get('books/' + id)
  return books
}

export const addBooks = (books) => {
  return apiClient.post('books/', books)
}
export const deleteBook = (id) => {
  const books = apiClient.delete('books/' + id)
  return books
}

export const updateBook = (id, bookData) => {
  return apiClient.put(`books/${id}`, bookData)
}
```

Réalisation

Explication sur chaque fonctionnalité technique demandée.

Une page d'accueil comprenant une explication de l'utilité du site ainsi que les cinq derniers ouvrages ajoutés (accès tout public). :
La liste des 5 derniers ouvrages est affichée grâce à un « fetch » via axios dans /books (chemin contenant les données.json)

Une page comprenant la liste des ouvrages par catégorie (accès tout public avec restrictions sur les liens). :

De base tout les livres sont affichés grâce a un « v-for » et leurs données sont « fetch » depuis le service getBooks de bookService. Pour le filtre, quand une catégorie est sélectionnée, sa valeur va dans une « ref », puis la méthode "filter()" est utilisée pour retourner le tableau avec seulement les livres dont la catégorie correspond.

Une page d'ajout d'un ouvrage (accès utilisateur).:
L'ajout d'un livre se fait via un form en utilisant des "v-model" sur chaque champ. Le "v-model" permet de syncroniser de façon bidirectionnelle les données du form et du code lié au form. Au moment de submit le form, une methode "submitForm()" est executée et envoie les données via le bookService qui fait une requête "post" sur les sur "db.json", puis les champs sont remis à 0.

Une page de modification d'un ouvrage (accès utilisateur pour ses ouvrages):
La modification d'un livre se fait presque de la même manière que l'ajout, en premier on utilise le même form, puis on charge les données du livre concerné dans le form avec la méthode "getBookById()" du bookService, au moment de submit la méthode "updateBook()" est appelée et fait un "put" sur lo localhost:3000/books/id_du_livre ce qui a pour effet de modifier les données.

Une page de suppression d'un ouvrage (accès utilisateur pour ses ouvrages):
Une page de suppression d'un ouvrage (accès utilisateur pour ses ouvrages): La suppression d'un livre est déclenchée par la méthode "handleDelete()" qui demande d'abord une confirmation de sécurité à l'utilisateur. Si l'action est validée, elle fait appel à la fonction "deleteBook()" du bookService qui exécute une requête "delete" sur "localhost:3000/books/id_du_livre". Une fois la suppression confirmée par le serveur, une alerte de succès est affichée et l'utilisateur est automatiquement redirigé vers la page "Catalogue" via le router.

Organisation du groupe et gestion du code (½ pages)

Pour organiser le travail de notre groupe, nous avons utilisé GitHub comme plateforme centrale pour le code et les documents du projet. La branche main contient la version stable et vérifiée de l'application, garantissant que toutes les fonctionnalités intégrées sont sécurisées.

Chaque personne à sa propre branche personnelle pour développer de nouvelles fonctionnalités, tester du code ou expérimenter des idées. Une fois qu'une tâche est terminée et validée, elle est merge dans la branche main après vérification, ce qui permet de maintenir la stabilité du projet tout en facilitant le développement parallèle.

Ce workflow Git favorise la collaboration et permet de résoudre rapidement les conflits éventuels rendant l'intégration des différentes parties du projet plus fluide et organisée. Grâce à cette approche, notre équipe a pu travailler efficacement tout en maintenant un code propre et structuré.

Conclusion

Le projet Passion Lecture nous a permis de mettre en pratique tout ce que nous avons appris dans le module 294, notamment avec Vue.js, la gestion des routes, les formulaires et la communication avec une API simulée. Au lieu de simples exercices, nous avons travaillé sur une application complète, avec plusieurs pages, une vraie structure et des fonctionnalités proches d'un site réel.

Ce projet nous a aidés à mieux comprendre comment organiser un frontend de manière propre et logique : découpage en composants, gestion de la navigation, structuration du code, validation des données, gestion des utilisateurs connectés, etc. Nous avons aussi appris à anticiper le fait qu'un backend sera ajouté plus tard, ce qui nous a obligés à réfléchir à une structure claire et évolutive dès le départ.

Le travail en groupe et l'utilisation de GitHub avec un tableau Kanban nous ont permis de mieux répartir les tâches et de suivre l'avancement du projet. Cela nous a donné un aperçu du fonctionnement d'un vrai projet en équipe, avec des branches, des tests et une intégration progressive du travail dans la branche principale.

Conclusion personnelle

Thomas Peltier

En conclusion, j'ai apprécié le travail d'équipe, que je trouve particulièrement bénéfique pour un projet comme celui-ci. Cela permet de répartir les tâches en fonction des compétences et des domaines dans lesquels chacun est le plus à l'aise. Cette organisation rend le travail plus efficace et plus agréable. En revanche, je n'ai pas particulièrement apprécié le Framework Vue.js, que je trouve assez complexe. De plus, je n'aime pas spécialement le JavaScript. Toutefois, le fait de travailler en équipe a rendu cette expérience plus intéressante et m'a permis d'aller au bout du projet dans de bonnes conditions.

Thomas Reynaud

En conclusion, j'ai trouvé que ce projet était très intéressant. J'ai beaucoup apprécié le fait de travailler en équipe : comme l'a dit Thomas Peltier plus haut, il est plus facile de se répartir les tâches, ce qui permet d'avancer vite et efficacement. J'ai particulièrement aimé Vue.js ; je trouve ce framework intrigant, d'autant que c'était une totale découverte

pour moi. Le début a été compliqué car nous n'avions aucune connaissance préalable, mais au fil des semaines, j'ai énormément appris. Ce fut une excellente expérience et j'ai hâte de développer le backend de cette application !

Ryan Läuppi

En conclusion c'était un projet sympa et intéressant à réaliser, je trouve que que ça change pas mal de travailler à plusieurs car on peut avancer plus vite mais il faut savoir s'approprier le code des autre et le comprendre ce qui peut prendre un peut de temps. Le début était un peu compliquer car on est parti de "0" sur l'API composition, mais au fur et à mesure du temps passé sur le projet je voyais une réelle progréssion de mes compétences en Vue.js ce que j'ai beaucoup apprécié.

Critique constructive sur la planification du projet (½ page)

La planification de ce projet était assez bien définie. Nous avons repris tous les objectifs du cahier des charges et les avons ajoutés dans le GitHub Project afin de pouvoir les organiser et suivre leur avancement. Cependant, ce que nous aurions pu améliorer serait de préciser davantage les étapes pour chaque objectif. En effet, notre planification restait parfois assez générale, ce qui rendait certaines tâches moins claires à réaliser. Une planification plus détaillée aurait probablement permis une meilleure organisation du travail.

Webographie / Bibliographique / Glossaire

Gemini.com

ChatGPT.com

Vue.js official documentation

MDN

w3schools.com

javascript.info

stackoverflow.com